

trie

梁宝烨

佛山市南海区石门中学

2025 年 7 月 16 日

目录

1 字典树

- 概念
- 字典树的建立
- 应用

2 01trie

- 概念
- 应用

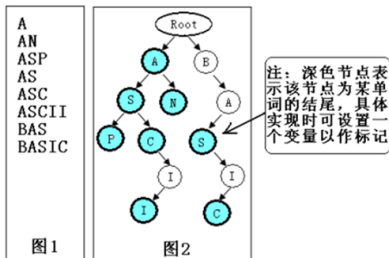
trie, 中文名字字典树, 是一种数据结构, 可以理解为是字符串的树形结构的桶。

trie, 中文名字字典树, 是一种数据结构, 可以理解为是字符串的树形结构的桶。

具体而言, 就是一棵边权是字母的树, 且以根节点为起点的某些树上的路径代表着一些被存储进来的字符串。

trie, 中文名字字典树, 是一种数据结构, 可以理解为是字符串的树形结构的桶。

具体而言, 就是一棵边权是字母的树, 且以根节点为起点的某些树上的路径代表着一些被存储进来的字符串。



示例代码如下：

```
1 char s[maxn];  
2 int tot, ch[maxn][26], cnt[maxn]; // tot是当前trie树的结点个  
   // 数, ch是儿子, cnt记录着相同的字符串有几个  
3 void insert(char *s) {  
4     int r = 0, d = s[0] - 'a', len = strlen(s);  
5     for (int i = 0; i < len; d = s[++i] - 'a') {  
6         if (!ch[r][d]) ch[r][d] = ++tot; // 如果先前没有这个  
           // 分支, 就先创建一个  
7         r = ch[r][d];  
8     } ++cnt[r];  
9 }
```

关于 trie 树的一些时空问题

假设 trie 树大小为 n , 字符集为 Σ :

关于 trie 树的一些时空问题

假设 trie 树大小为 n , 字符集为 Σ :

- 每个点都开个 Σ 大小的儿子数组, 则可以 $O(1)$ 新建儿子, $O(1)$ 向下找儿子, 但空间是 $O(n \cdot \Sigma)$ 。

关于 trie 树的一些时空问题

假设 trie 树大小为 n , 字符集为 Σ :

- 每个点都开个 Σ 大小的儿子数组, 则可以 $O(1)$ 新建儿子, $O(1)$ 向下找儿子, 但空间是 $O(n \cdot \Sigma)$ 。
- 每个点使用 vector 存儿子, 则也是 $O(1)$ 新建儿子, 只能 $O(\Sigma)$ 向下找儿子, 但空间是 $O(n)$ 。

关于 trie 树的一些时空问题

假设 trie 树大小为 n , 字符集为 Σ :

- 每个点都开个 Σ 大小的儿子数组, 则可以 $O(1)$ 新建儿子, $O(1)$ 向下找儿子, 但空间是 $O(n \cdot \Sigma)$ 。
- 每个点使用 vector 存儿子, 则也是 $O(1)$ 新建儿子, 只能 $O(\Sigma)$ 向下找儿子, 但空间是 $O(n)$ 。
- 每个点使用 map 存儿子, 则只能 $O(\log \Sigma)$ 新建儿子, 但可以 $O(\log \Sigma)$ 向下找儿子, 空间是 $O(n)$ 。

- 只枚举字符串 i ，用 $\text{len}(i)$ 的时间，能够匹配完所有字符串 j (将那些 j 全部插入 trie 树)。

- 只枚举字符串 i ，用 $\text{len}(i)$ 的时间，能够匹配完所有字符串 j (将那些 j 全部插入 trie 树)。
- 匹配的同时也可以得知字符串之间的前缀关系。

- 只枚举字符串 i , 用 $\text{len}(i)$ 的时间, 能够匹配完所有字符串 j (将那些 j 全部插入 trie 树)。
- 匹配的同时也可以得知字符串之间的前缀关系。
- 更多的应用是作为 AC 自动机的一部分。

练习

- P8306 【模板】字典树
- [ABC287E] Karuta
- [ABC403E] Forbidden Prefix

01trie 是一种特殊的 trie, 它的 $\Sigma = \{0, 1\}$ 。

01trie 是一种特殊的 trie, 它的 $\Sigma = \{0, 1\}$ 。
为什么要单独拿出来讲, 因为它在处理异或问题的时候非常方便。

01trie 是一种特殊的 trie, 它的 $\Sigma = \{0, 1\}$ 。

为什么要单独拿出来讲, 因为它在处理异或问题的时候非常方便。

这样子看 01trie 数据结构就是一个用来存储数的“桶”(其他存储数的“桶”还有数组, 值域线段树等)。

设 V 是值域, 则 01trie 插入的时间是 $O(\log V)$, 若插入 n 个数, 空间 $O(n \log V)$ 。

设 V 是值域，则 01trie 插入的时间是 $O(\log V)$ ，若插入 n 个数，空间 $O(n \log V)$ 。
一般存储的时候是从高位到低位。

设 V 是值域, 则 01trie 插入的时间是 $O(\log V)$, 若插入 n 个数, 空间 $O(n \log V)$ 。

一般存储的时候是从高位到低位。

示例代码如下:

```
1 int tot, ch[maxn][2];
2 void insert(int x) {
3     for (int i = 30; i >= 0; i--) {
4         int d = x >> i & 1;
5         if (!ch[r][d]) ch[r][d] = ++tot;
6         r = ch[r][d];
7     }
8 }
```

求点对的最大/小异或值

给定 n 个数 $a_1, a_2, \dots, a_n$, 多次询问, 每次问一个数 x 与这些数异或起来最大/小值。

求点对的最大/小异或值

给定 n 个数 $a_1, a_2, \dots, a_n$, 多次询问, 每次问一个数 x 与这些数异或起来最大/小值。

做法很简单, 先将这 n 个数插入到 01trie 中, 然后每次询问 x , 贪心地在 01trie 上二分即可。

进行一些特殊的修改操作

- 01trie 支持全体数异或 x 。

进行一些特殊的修改操作

- 01trie 支持全体数异或 x 。
其实就是看 x 的哪些位是 1，然后翻转 01trie 对应层的所有点的左右儿子，可以打懒标记而不实际翻转。

进行一些特殊的修改操作

- 01trie 支持全体数异或 x 。
其实就是看 x 的哪些位是 1，然后翻转 01trie 对应层的所有点的左右儿子，可以打懒标记而不实际翻转。
- 01trie 还支持全体数加 1。

进行一些特殊的修改操作

- 01trie 支持全体数异或 x 。
其实就是看 x 的哪些位是 1，然后翻转 01trie 对应层的所有点的左右儿子，可以打懒标记而不实际翻转。
- 01trie 还支持全体数加 1。
这个很巧妙，我们需要更改一下 01trie 的存储，变成从低位到高位。

进行一些特殊的修改操作

- 01trie 支持全体数异或 x 。
其实就是看 x 的哪些位是 1，然后翻转 01trie 对应层的所有点的左右儿子，可以打懒标记而不实际翻转。
- 01trie 还支持全体数加 1。
这个很巧妙，我们需要更改一下 01trie 的存储，变成从低位到高位。
那么 $+1$ 对于最低位来说，就相当于翻转。但是有进位，会让次低位 $+1$ 。

进行一些特殊的修改操作

- 01trie 支持全体数异或 x 。
其实就是看 x 的哪些位是 1，然后翻转 01trie 对应层的所有点的左右儿子，可以打懒标记而不实际翻转。
- 01trie 还支持全体数加 1。
这个很巧妙，我们需要更改一下 01trie 的存储，变成从低位到高位。
那么 $+1$ 对于最低位来说，就相当于翻转。但是有进位，会让次低位 $+1$ 。
但你会发现只有原本是 1 的 $+1$ 了才会进位。

进行一些特殊的修改操作

- 01trie 支持全体数异或 x 。
其实就是看 x 的哪些位是 1，然后翻转 01trie 对应层的所有点的左右儿子，可以打懒标记而不实际翻转。
- 01trie 还支持全体数加 1。
这个很巧妙，我们需要更改一下 01trie 的存储，变成从低位到高位。
那么 $+1$ 对于最低位来说，就相当于翻转。但是有进位，会让次低位 $+1$ 。
但你会发现只有原本是 1 的 $+1$ 了才会进位。
因此 $+1$ 操作相当于让原本的 01trie 的一条向右的链全部翻转左右儿子。

练习

CF282E Sausage Maximization

CF842D Vitya and Strange Lesson

P6018 [Ynoi2010] Fusion tree

思考题：a,b 两个人要玩取数游戏，有 $2n$ 个数。

每一轮中，每个人都会取出一个数（之后不能再取），假设取出的是 x, y ，那么这一轮分数就是 $x \oplus y$ 。

总共进行 n 轮，a 先手。a 想让所有轮分数的最大值尽可能大，b 想让所有轮分数的最大值尽可能小。

求最终所有轮分数的最大值是多少。 $n \leq 10^5$ ，值域 $[0, 2^{30})$ 。